PROYECTO DATA ANALYTICS

"ANÁLISIS PREDICTIVO PARA LA OPTIMIZACIÓN DE RECURSOS HOSPITALARIOS"

Grupo4

Javier Troyano

Loreto Redondo

Mapa Mental

- Pasos a seguir -

Datos. Importación y limpieza

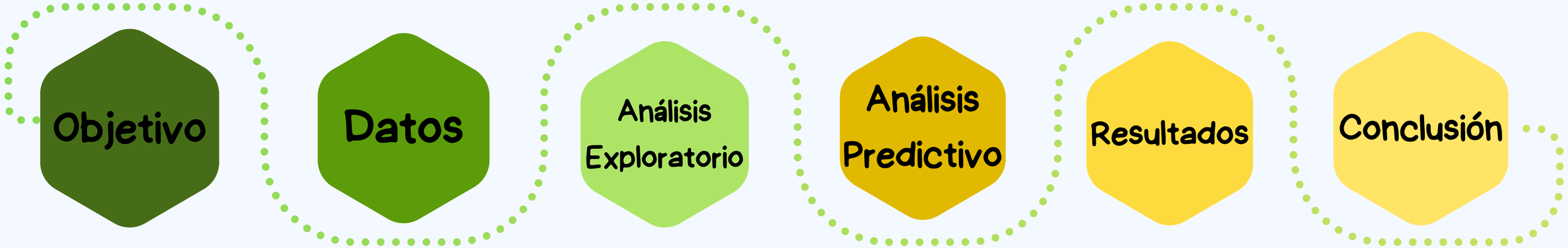
Fase 1. Analizamos datos y procedemos a resolver nulos, realizar agrupaciones y eliminar e incluir columnas.

Análisis Predictivo

Preparamos los datos, entrenamos los modelos y los evaluamos.

Conclusiones

En esa última fase damos respuesta a nuestro objetivo a través de observaciones claras. Analizamos las limitaciones y futuras mejoras.



Objetivo

Etapa donde nos planteamos queremos predecir, cómo lo vamos a enfocar, en que nos vamos a basar.

Análisis Exploratorio

Utilizamos diferentes graficas para entender las columnas, relaciones y tendencias entre los valores.

Resultados

Comparamos los modelos creados. Valoramos acurracy y área bajo la curva.

Objetivo

Datos

Análisis Exploratorio

Análisis Predictivo

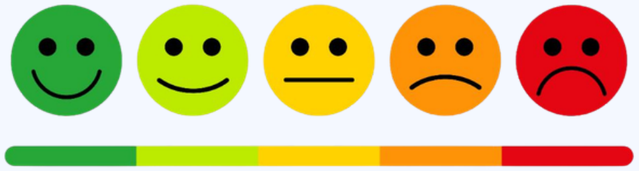
Resultados

Conclusión

URGENT

Hospitales en alerta

La saturación hospitalaria es uno de los mayores retos de los sistemas sanitarios. La capacidad de predecir la duración de estancia desde el momento de admisión permitiría optimizar recursos, gestionar camas y reducir tiempos de espera.



Objetivo

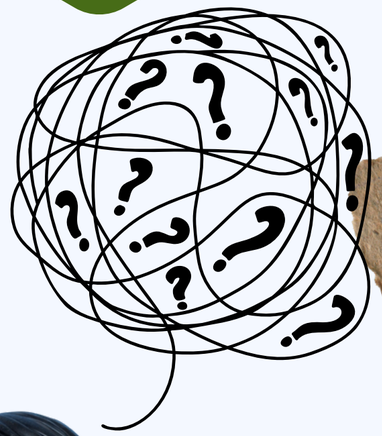
Datos

Análisis Exploratorio

Análisis Predictivo

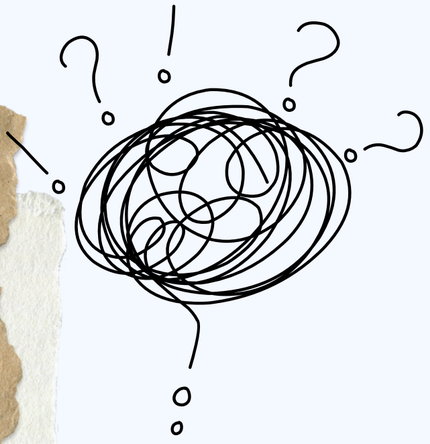
Resultados

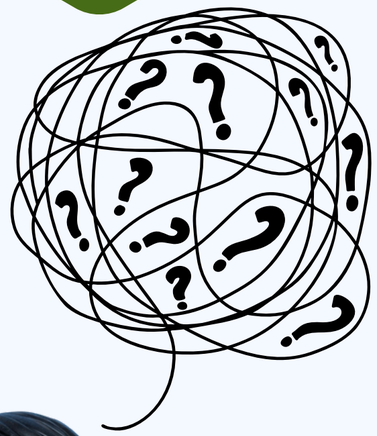
Conclusión



¿CUÁNTO TIEMPO HACE USO UN PACIENTE DE UNA CAMA ?

¿DE QUÉ DEPENDE ESA ESTANCIA EN UN HOSPITAL?





EL OBJETIVO DEL PROYECTO ES REALIZAR UN PROGRAMA QUE NOS INDIQUE, SEGÚN LAS VARIABLES APORTADAS POR LA BASE DE DATOS, EL TIEMPO DE ESTANCIA DE UN PACIENTE EN EL HOSPITAL.

Objetivo

Datos

Análisis Exploratorio

Análisis Predictivo

Resultados

Conclusión

FASE 1

1. Análisis DataSet
2. Verificación valores nulos NULL
3. Agregación columnas
4. Realización de agrupaciones
5. Eliminación de columnas

Objetivo

Datos

Análisis Exploratorio

Análisis Predictivo

Resultados

Conclusión

Vamos a limpiar todos estos datos....

case_id	0.00
Hospital_code	0.00
Hospital_type_code	0.00
City_Code_Hospital	0.00
Hospital_region_code	0.00
Available Extra Rooms in Hospital	0.00
Department	0.00
Ward_Type	0.00
Ward_Facility_Code	0.00
Bed Grade	0.04
patientid	0.00
City Code Patient	1.42
Type of Admission	0.00
Severity of Illness	0.00
Visitors with Patient	0.00
Age	0.00
Admission_Deposit	0.00
Stay	0.00

COMPROBAMOS NULOS

case_id	0.0
Hospital_code	0.0
Hospital_type_code	0.0
City_Code_Hospital	0.0
Hospital_region_code	0.0
Available Extra Rooms in Hospital	0.0
Department	0.0
Ward_Type	0.0
Ward_Facility_Code	0.0
Bed Grade	0.0
patientid	0.0
City_Code_Patient	0.0
Type of Admission	0.0
Severity of Illness	0.0
Visitors with Patient	0.0
Age	0.0
Admission_Deposit	0.0
Stay	0.0

ELIMINAMOS NULOS

Objetivo

Datos

Análisis Exploratorio

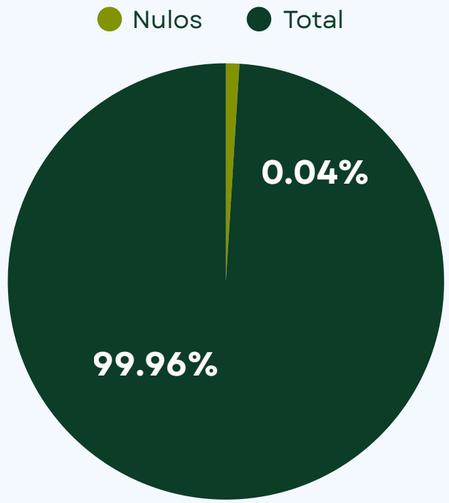
Análisis Predictivo

Resultados

Conclusión

COLUMNA

Bed Grade



Eliminamos Valores Nulos

```
df_td_copy1 = df_train_data.copy()

df_td_copy1 = df_td_copy1.dropna(subset = ["Bed Grade"])
```

delete

Objetivo

Datos

Análisis Exploratorio

Análisis Predictivo

Resultados

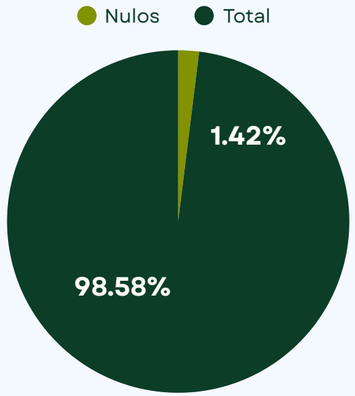
Conclusión

Comprobamos si los pacientes han ido más de una vez al hospital

Comparamos si los pacientes con NaN han hecho otras visitas en las que se les haya incluido el código

COLUMNA

CITY_CODE_PATIENT



Incluimos Categoría "Other"

```
df_prueba["Numero_nulos_city"] = df_prueba["City_Code_Patient"].isna().groupby(df_prueba["patientid"]).transform("sum")
df_prueba["Numero_no_nulos_city"] = df_prueba["City_Code_Patient"].notna().groupby(df_prueba["patientid"]).transform("sum")
df_prueba.head(2)
```

Generamos código para verificar si en alguna visita del paciente quedó registrado el CODE

```
filtro = (df_prueba["Numero_nulos_city"]> 0) & (df_prueba["Numero_no_nulos_city"] > 0)
df_prueba.loc[filtro, :]
```

```
df_td_copy2 = df_td_copy1.copy()
df_td_copy2["City_Code_Patient"] = df_td_copy2["City_Code_Patient"].fillna("other")
```

Creamos la nueva categoría para los valores nulos

execute

COLUMNAS

Agrupamos "Age" y "Stay"

```
df_td_copy2["Age"] =
df_td_copy2["Age"].replace({
    "0-10" : "0-20",
    "11-20" : "0-20",
    "21-30": "21-50",
    "31-40" : "21-50",
    "41-50" : "21-50",
    "51-60": "51-70",
    "61-70" : "51-70",
    "71-80" : "71-100",
    "81-90" : "71-100",
    "91-100": "71-100"
})
```

```
df_td_copy2["Stay"] =
df_td_copy2["Stay"].replace({
    "0-10": "Hasta 10 dias",
    "11-20": "Menos de 40 dias",
    "21-30": "Menos de 40 dias",
    "31-40": "Menos de 40 dias",
    "41-50": "Más de 40 días",
    "51-60": "Más de 40 días",
    "61-70": "Más de 40 días",
    "71-80": "Más de 40 días",
    "81-90" : "Más de 40 días",
    "91-100" : "Más de 40 días",
    "More than 100 Days": "Más de 40 días"
})
```

Creamos columna : Visitas por Paciente

```
df_td_copy2["Numero_registros_paciente"] =
df_td_copy2["patientid"].map(df_td_copy2["patientid"].value_counts())
```

Eliminamos columnas

```
df_td_copy3 =
df_td_copy3.drop(columns = ["Ward_Facility_Code",
                            "Visitors with Patient",
                            "Admission_Deposit"])
```

Objetivo

Datos

Análisis Exploratorio

Análisis Predictivo

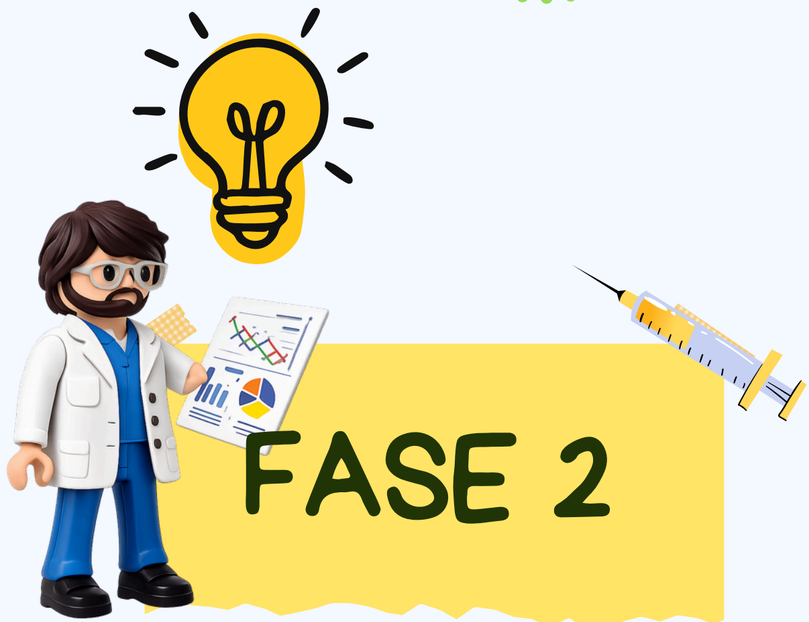
Resultados

Conclusión

El análisis exploratorio será explicado y mostrado a través de

Power BI

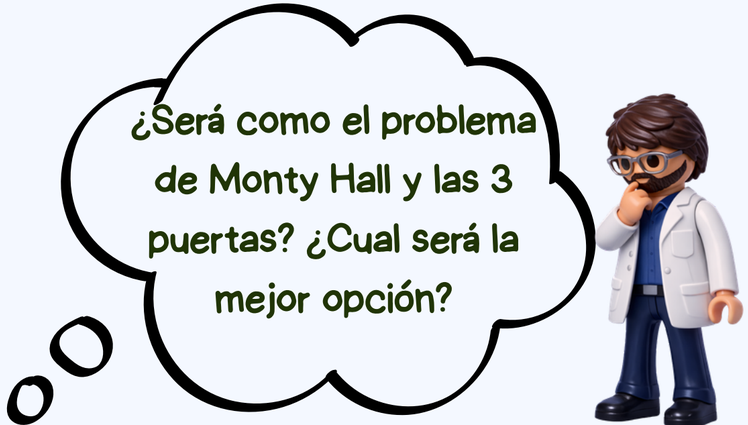
Objetivo | Datos | Análisis Exploratorio | **Análisis Predictivo** | Resultados | Conclusión



FASE 2

Desarrollo y comparación de distintos modelos predictivos para estimar la duración de estancia hospitalaria desde el momento de admisión.

- Regresión Logística
- Random Forest
- XGBoost



Pasos conjuntos para la creacción de modelos

Eliminamos columnas en R.Logística

```
df_train_data_clean =
df_train_data_clean.drop(columns = [
    "Unnamed: 0", "case_id", "patientid", "Hospital_code",
    "City_Code_Hospital", "City_Code_Patient"])
```

Eliminamos columnas R.Forest y XGBoots

```
df_train_data_clean = df_train_data_clean.drop(columns = [
    "Unnamed: 0", "case_id", "patientid"])
```

Separamos en TRAIN y TEST

```
X = df_train_data_clean.drop(columns = "Stay")
y = df_train_data_clean["Stay"]

X_train, X_test, y_train, y_test,  = train_test_split(
    X, y,
    train_size = 0.75,
    test_size = 0.25,
    random_state = 10
)
```

Generamos Dummies

```
X_train = pd.get_dummies(
    X_train,
    columns= [
        "Hospital_type_code",
        "Hospital_region_code",
        "Department",
        "Ward_Type",
        "Bed Grade",
        "Type of Admission",
        "Severity of Illness",
        "Age",
    ],
    drop_first = True
)
```

```
X_test = pd.get_dummies(
    X_test,
    columns= [
        "Hospital_type_code",
        "Hospital_region_code",
        "Department",
        "Ward_Type",
        "Bed Grade",
        "Type of Admission",
        "Severity of Illness",
        "Age",
    ],
    drop_first = True
)
```

Regresión Logística

Selección de variables

```
rfe = RFE(estimator = LogisticRegression(max_iter = 100000),
          n_features_to_select = 20
)
rfe.fit(X_train, y_train)
```

```
X_train_final = X_train.loc[:, rfe.support_]
X_test_final = X_test.loc[:, rfe.support_]
```

Creacción de modelo

```
logreg = LogisticRegression(max_iter = 10000000)
logreg.fit(X_train_final, y_train)
logreg.score(X_test_final, y_test)
y_pred = logreg.predict(X_test_final)
```

Validación

```
matriz_confusion = confusion_matrix(y_test, y_pred)
pd.DataFrame({
    "y_real": y_test,
    "y_pred": y_pred
}).groupby(["y_pred", "y_real"]).size()
```

```
(TN + TP) / (TN + TP + FN + FP) ## Accuracy.
TP / (TP + FN) ## sensibilidad.
TN / (TN + FP) ## especificidad.
```

Random Forest

Generación de modelo-restringido

```
for i in range(1,20):
    modelo = DecisionTreeClassifier(max_depth = i)
    modelo.fit(X_train, y_train)

    y_pred = modelo.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    print(i, "->", np.round(acc,3))
```

Comprobamos cuál es el rendimiento óptimo para el modelo final

Creacción de modelo

```
modelo = RandomForestClassifier(max_depth = 10)
modelo.fit(X_train, y_train)
```

Validación

```
y_pred = modelo.predict(X_test)
accuracy_score(y_test, y_pred)
```

Columna "Stay" a valor numérico

```
df_train_data_clean["Stay"] = df_train_data_clean["Stay"].replace({
    "Menos de 40 dias": 0,
    "Entre 40 y 70 días": 1,
    "Más de 70 días": 2
})
```

También comprobamos a través de modelo restringido el rendimiento óptimo

Creacción de modelo

```
modelo = XGBClassifier(max_depth = 10)
modelo.fit(X_train, y_train)
```

Validación

```
y_pred = modelo.predict(X_test)
accuracy_score(y_test, y_pred)
```

XGBoost

Objetivo | Datos | Análisis Exploratorio | Análisis Predictivo | **Resultados** | Conclusión

REGRESIÓN LOGÍSTICA

Accuracy = exactitud

```
(TN + TP) / (TN + TP + FN + FP)
```

```
np.float64(0.8304479402382835)
```

RANDOM FOREST

Accuracy = exactitud

```
y_pred = modelo.predict(X_test)
accuracy_score(y_test, y_pred)
```

```
0.7477067678620793
```

XGBOOTS

Accuracy = exactitud

```
y_pred = modelo.predict(X_test)
accuracy_score(y_test, y_pred)
```

```
0.7762936342388982
```

Calculamos el AUC: Área bajo la curva

```python
from sklearn.metrics import roc_auc_score

proba = modelo.predict_proba(X_test)
auc_ovr_macro = roc_auc_score(y_test, proba, multi_class="ovr", average="macro")
auc_ovr_weighted = roc_auc_score(y_test, proba, multi_class="ovr", average="weighted")

print("AUC OvR macro:", auc_ovr_macro)
print("AUC OvR weighted:", auc_ovr_weighted)
```

```python
from sklearn.metrics import roc_auc_score

proba = logreg.predict_proba(X_test_final)
auc_ovr_macro = roc_auc_score(y_test, proba, multi_class="ovr", average="macro")
auc_ovr_weighted = roc_auc_score(y_test, proba, multi_class="ovr", average="weighted")

print("AUC OvR macro:", auc_ovr_macro)
print("AUC OvR weighted:", auc_ovr_weighted)
```

REGRESIÓN LOGÍSTICA

AUC

```
AUC OvR macro: 0.6601706649946434
AUC OvR weighted: 0.6565085836125901
```

RANDOM FOREST

AUC

```
AUC OvR macro: 0.8330914261401928
AUC OvR weighted: 0.8322265449497653
```

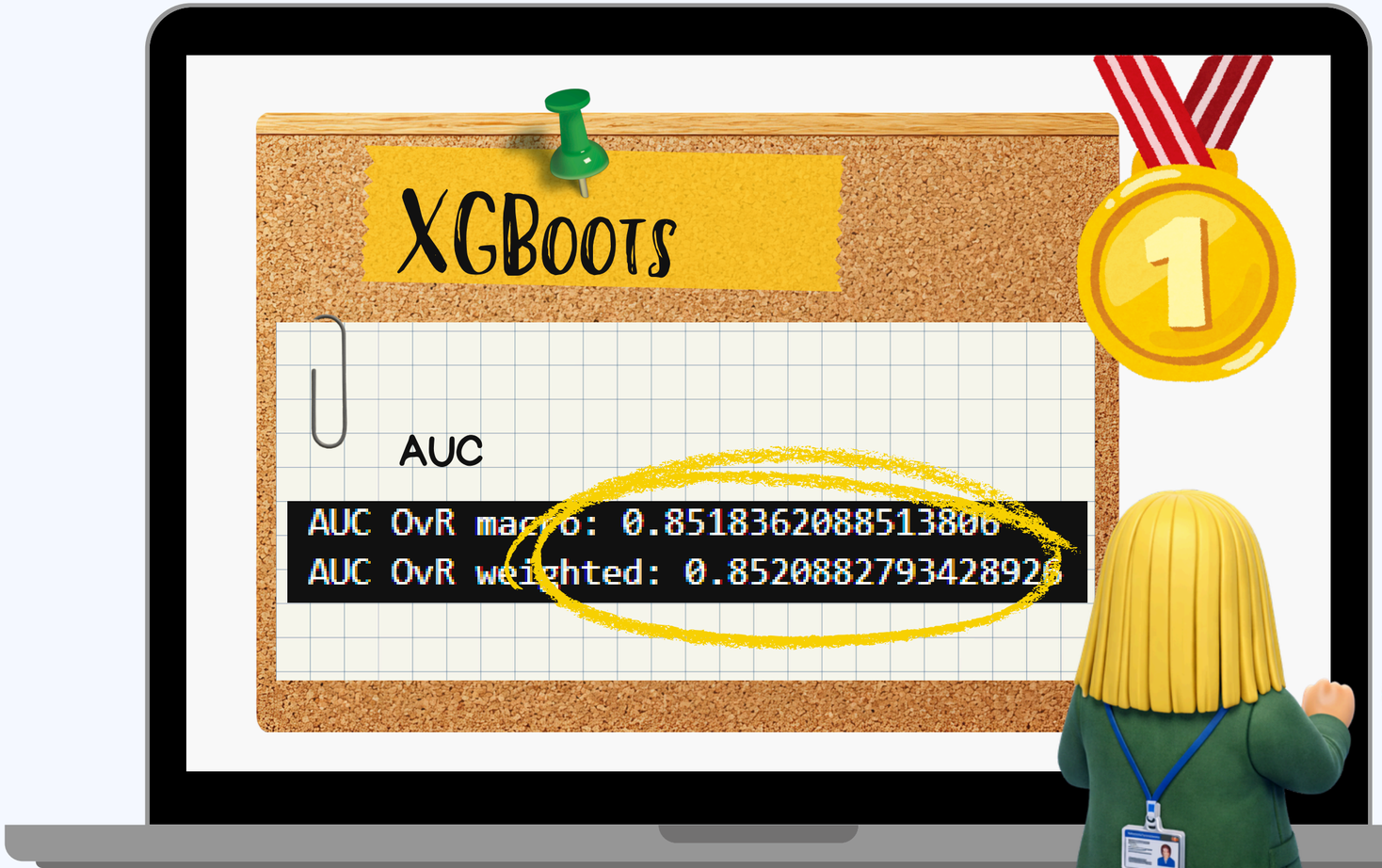
XGBOOTS

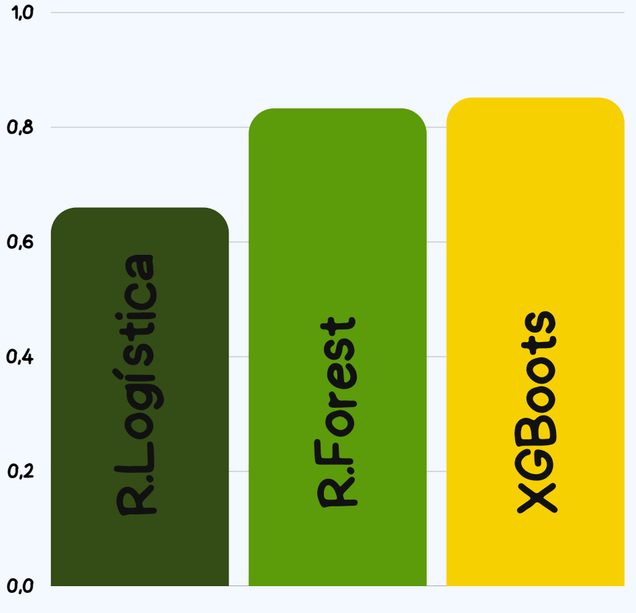
AUC

```
AUC OvR macro: 0.8518362088513806
AUC OvR weighted: 0.8520882793428926
```

Objetivo | Datos | Análisis Exploratorio | Análisis Predictivo | Resultados | Conclusión

MODELO ELEGIDO: XGBOOTS
¿Por qué?





Objetivo

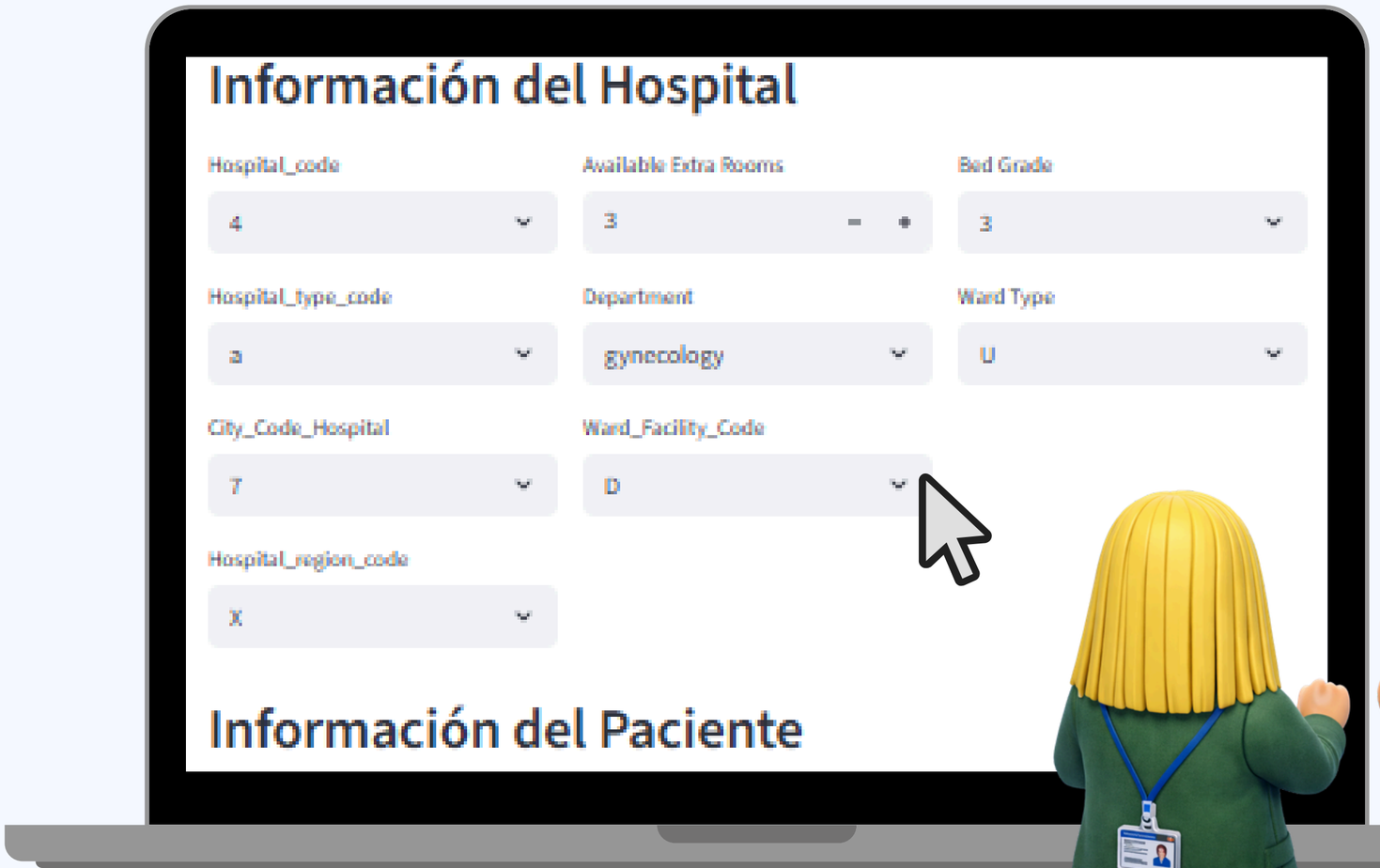
Datos

Análisis Exploratorio

Análisis Predictivo

Resultados

Conclusión



MODELO ELEGIDO: XGBOOTS

Una vez elegido el modelo, generamos programa de predicción a través de

Streamlit

Objetivo

Datos

Análisis Exploratorio

Análisis Predictivo

Resultados

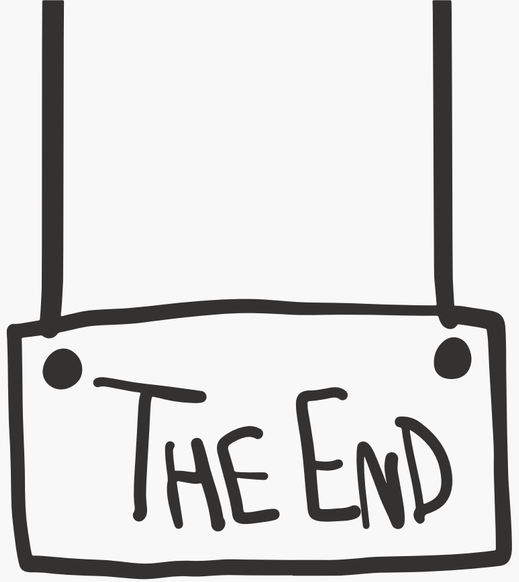
Conclusión

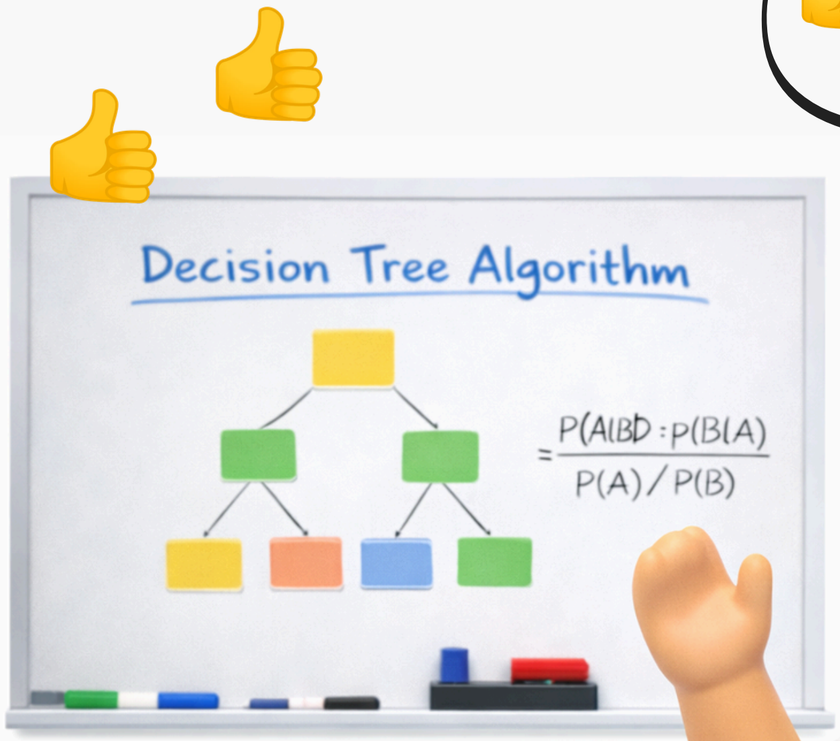
PRÓXIMOS PASOS:

Ampliar base con datos más específicos del caso registrado.
Por ejemplo: enfermedad o traumatismo.

Generar una base de datos con situaciones reales en una comunidad de España.

AGRADECIMIENTOS





Chicos, ponganme un 👍 si están todos bien hasta acá

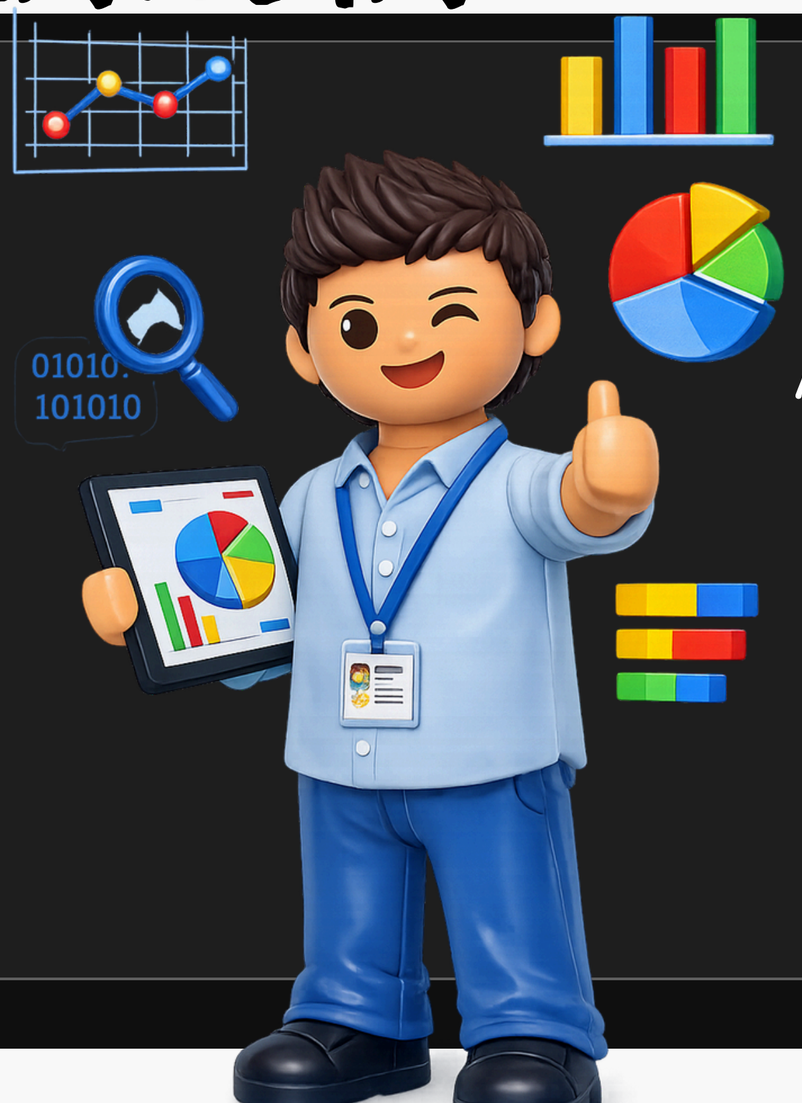
CONSEJO DE ANALÍSTA

```
[ ]: import numpy as np
     import pandas as pd
     import matplotlib.pyplot as plt
     import statsmodels.formula.api as smf
     import seaborn as sns
     from sklearn.model_selection import train_test_split

     from sklearn.linear_model import LogisticRegression
     from sklearn.feature_selection import RFE

     from sklearn.ensemble import RandomForestClassifier
     from sklearn.metrics import accuracy_score

     from sklearn import tree
     from sklearn.tree import DecisionTreeClassifier
     from sklearn.tree import plot_tree
     import xgboost as xgb
     from xgboost import XGBClassifier
     import joblib
     from sklearn.metrics import mean_squared_error
```



Acuérdate siempre de ejecutar bien todas las librerías